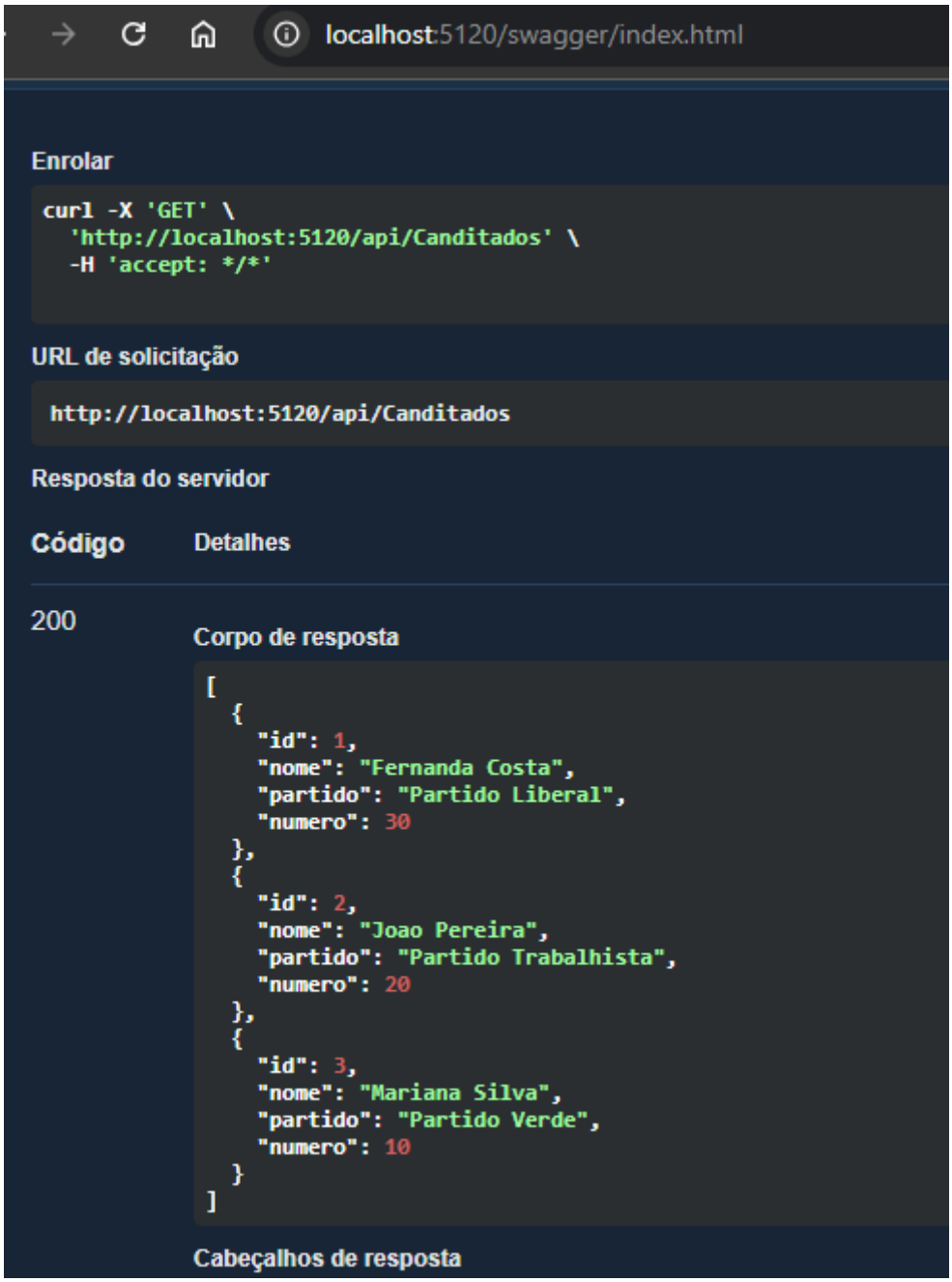
1.

Cadastre 3 candidatos no POST.
Depois execute o GET da listagem e confirme se os 3 aparecem.



Pare a aplicação e rode novamente.
Execute o GET outra vez.

2. Resposta da questão sobre banco In-Memory.

R: Não, os dados não permanecem após reiniciar a aplicação.
Isso acontece porque o banco de dados In-Memory armazena as

informações apenas na memória RAM, que é temporária. Quando a aplicação é encerrada, todos os dados são apagados.

Esse tipo de banco é útil para testes e desenvolvimento rápido, mas em sistemas reais pode ser um problema, pois não garante persistência dos dados. Por outro lado, pode ser uma solução em cenários onde não é necessário salvar informações permanentemente, como testes automatizados.

3. Teste a semântica HTTP. No POST, envie um objeto vazio no Swagger. Com a configuração atual, o comportamento esperado é 400 Bad Request, porque a API está com [ApiController] e o modelo exige campos obrigatórios para funcionar corretamente. No DELETE, teste um ID inexistente, como 99. No seu código atual, o retorno esperado é 404 Not Found, porque o controller procura o candidato e, se não encontrar, devolve “não encontrado”.

localhost:5120/swagger/index.html

Enrolar

```
curl -X 'DELETE' \
  'http://localhost:5120/api/Canditados/99' \
  -H 'accept: */*'
```

URL de solicitação

```
http://localhost:5120/api/Canditados/99
```

Resposta do servidor

Código	Detalhes
404 *Sem documentação*	Erro: Não encontrado Corpo de resposta


```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",
  "title": "Not Found",
  "status": 404,
  "traceId": "00-9db392fc892db8bffb70ac0ec067d446-bd5ee2564dae60ec-00"
}
```


Cabeçalhos de resposta

```
content-type : application/problem+json; charset=utf-8
data : Sexta-feira, 03 de abril de 2026 02:29:57 GMT
Servidor : Kestrel
codificação de transferência : chunked
```

Respostas

Código	Descrição
200	OK

localhost:5120/swagger/index.html

URL de solicitação

```
http://localhost:5120/api/Canditados
```

Resposta do servidor

Código	Detalhes
500 *Sem documentação*	Erro: Solicitação inválida Corpo de resposta


```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Nome": [
      "The Nome field is required."
    ],
    "Partido": [
      "The Partido field is required."
    ]
  },
  "traceId": "00-5d4354a5aa2ce227c9c80b06c70e705f-5618d6d65be31fe7-00"
```

4.

Semântica HTTP

POST com objeto vazio

Resposta:

Ao enviar um objeto vazio, a API deve retornar **400 Bad Request**, pois os dados enviados são inválidos ou incompletos.

DELETE com ID inexistente

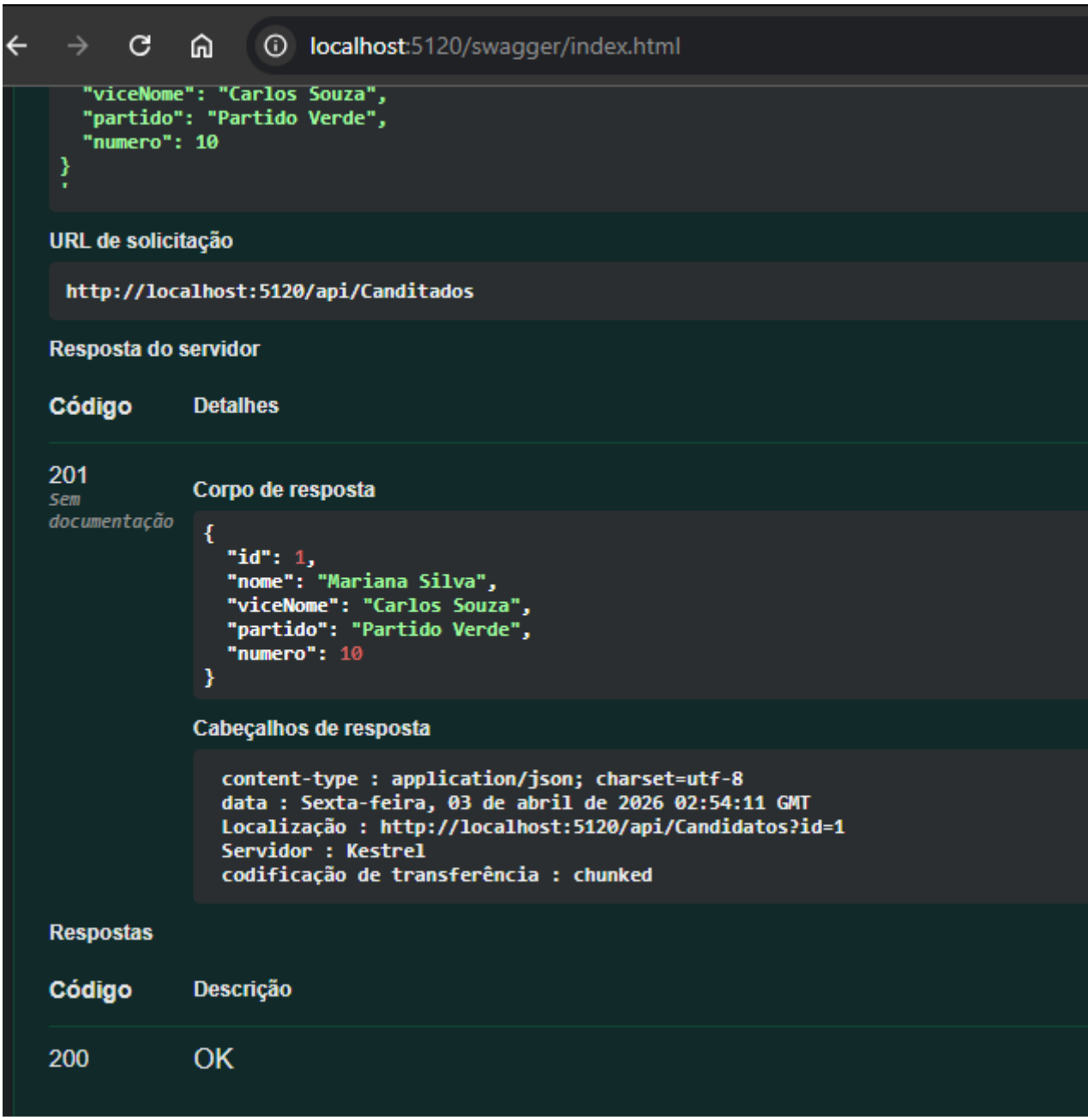
Resposta:

O comportamento ideal é retornar **404 Not Found**, pois o recurso que se deseja deletar não existe.

Em sistemas distribuídos robustos, retornar 404 é mais adequado, pois informa corretamente ao cliente que o recurso não foi encontrado.

5.

Faça a alteração do modelo.
Abra Candidato.cs.
Adicione o novo campo ViceNome.



6. Validação de Regra de Negócio (O "Pulo do Gato"):
No método Post (Create) do seu Controller, adicione uma validação: o Regra: Não podem existir dois candidatos com o mesmo número de eleição. o Dica de Código: Use _context.Candidatos.Any(c => c.Numero == novoCandidato.Numero) antes de salvar. Se já existir, retorne um BadRequest("Número já cadastrado").

localhost:5120/swagger/index.html

Respostas

Enrolar

```
curl -X 'POST' \
  'http://localhost:5120/api/Canditados' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 2,
  "nome": "Joao Pereira",
  "viceNome": "Ana Lima",
  "partido": "Partido Trabalhista",
  "numero": 10
}'
```

URL de solicitação

```
http://localhost:5120/api/Canditados
```

Resposta do servidor

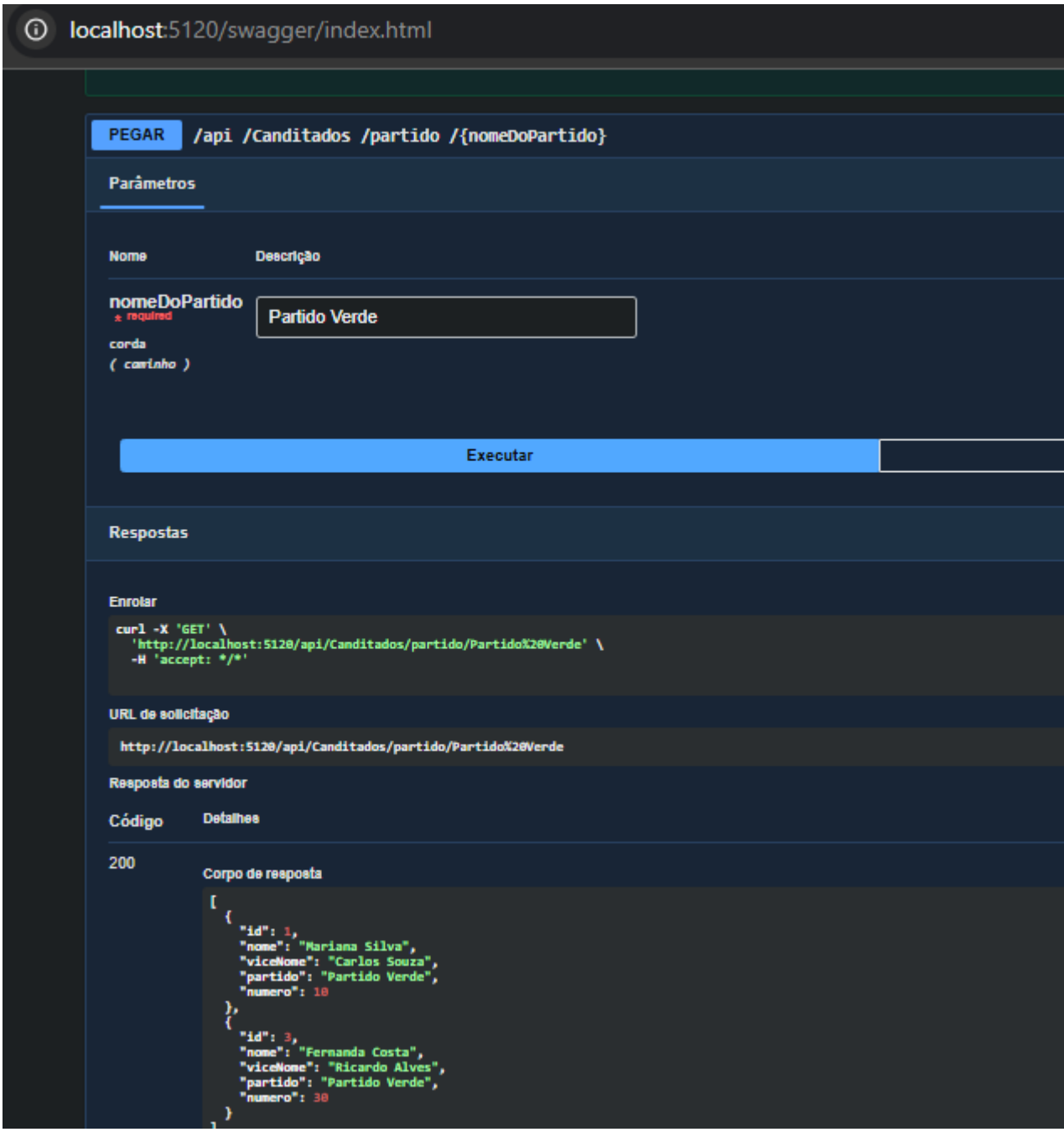
Código	Detalhes

201
Sem documentação
	Erro: Solicitação inválida
	Corpo de resposta
	`Número já cadastrado`
	Cabeçalhos de resposta
	`content-type : application/json; charset=utf-8`
	`data : Sexta-feira, 03 de abril de 2026 03:15:46 GMT`
	`Servidor : Kestrel`
	`codificação de transferência : chunked`

Respostas

Código	Descrição
200	OK

7. Desafio de Busca (Filtro): Crie um novo método GET no Controller que aceite um parâmetro de busca: api/candidatos/partido/{nomeDoPartido}. o Este método deve retornar apenas os candidatos daquele partido específico. o Dica: Use _context.Candidatos.Where(c => c.Partido == nomeDoPartido).ToList().



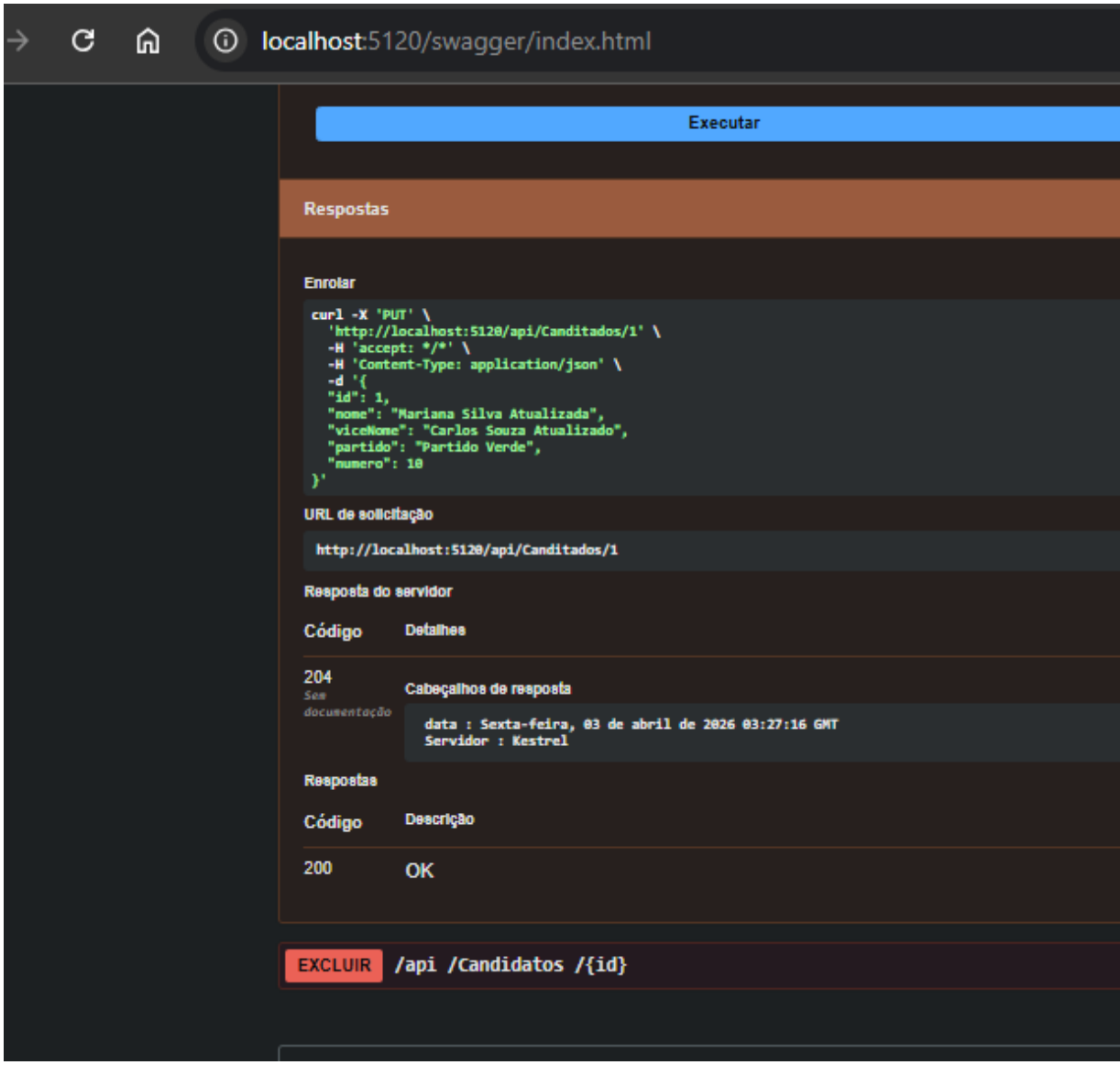
8. Idempotência na Prática: o Execute o método DELETE para o ID 1 duas vezes seguidas. O resultado da segunda execução foi diferente da primeira no que diz respeito ao estado do banco de dados? Isso prova que o DELETE é um método idempotente? Justifique.

Resposta:

Ao executar o DELETE duas vezes para o mesmo ID, a primeira execução remove o candidato e a segunda não altera o estado do sistema.

Isso demonstra que o método DELETE é idempotente, pois executar a mesma operação múltiplas vezes resulta no mesmo estado final do sistema.

PUT



DELET

localhost:5120/swagger/index.html

EXCLUIR /api /Candidatos /{id}

Parâmetros

Nome	Descrição
eu ia * required inteiro ($int32) (caminho)	1

Executar

Respostas

Enrolar

```
curl -X 'DELETE' \
  'http://localhost:5120/api/Canditados/1' \
  -H 'accept: */*'
```

URL de solicitação

```
http://localhost:5120/api/Canditados/1
```

Resposta do servidor

Código	Detalhes
204 Sem documentação	Cabeçalhos de resposta data : Sexta-feira, 03 de abril de 2026 03:29:58 GMT Servidor : Kestrel

Respostas

Código	Descrição
200	OK